

Migrate VSession Server Script

1. Purpose

This script migrates VSession.server (and session_id where applicable) between Vonage-style and OpenVidu-style values for eligible future scheduled sessions for a single provider (vsessions.prov_id = --provider-id). Optionally you can limit rows to sessions created by specific client contacts via --client-emails, described in Section 3.1. It writes structured logs to the console and to a log file so operators can audit what would change (dry run) or what was changed (apply).

The implementation lives at scripts/python/migrate_vsession_server.py, relative to the repository root. Run it as a module so that relative imports such as script_utils resolve correctly:

```
poetry run python -m scripts.python.migrate_vsession_server ...
```

2. Eligibility Filters

A row is migrated only when all of the following are true.

Filter	Meaning
scheduled_at > now (UTC)	Session is scheduled in the future (get_utc_now()).
started_at IS NULL	Call has not started.
ended_at IS NULL	Call has not ended.
call_type == SCHEDULED	Scheduled call type (SCHEDULED constant).
state IN (...)	interpreter_sourced or no_interpreter_sourced (INTERPRETER_SOURCED_STATE, NO_INTERPRETER_SOURCED_STATE).
vsessions.prov_id	Must equal --provider-id (providers.prov_id).
vsession_created_by	If --client-emails is set, created_by_id must be one of the resolved contacts.cont_id rows for those emails and the same prov_id. Sessions with no matching creator are out of scope.

Additional filters depend on migration direction, described in Section 4. The base query below is the starting point, and the direction-specific server filters are applied on top of it:

```
q = db.session.query(VSession).filter(
    VSession.scheduled_at > get_utc_now(),
    VSession.started_at.is_(None),
    VSession.ended_at.is_(None),
    VSession.call_type == SCHEDULED,
    VSession.state.in_(
        (
            INTERPRETER_SOURCED_STATE,
            NO_INTERPRETER_SOURCED_STATE,
        )
    ),
    VSession.prov_id == provider_id,
)
if created_by_contact_ids is not None:
    q = q.filter(VSession.created_by_id.in_(created_by_contact_ids))
```

The --provider-id value is validated before the query runs. It must be a positive integer that corresponds to an existing providers row:

```
def _validate_provider_id(provider_id: int) -> None:
    if provider_id <= 0:
        _fail("--provider-id must be a positive integer (not 0).")
    if db.session.query(Provider.id).filter(Provider.id == provider_id).first() is None:
        _fail(f"No provider row for prov_id={provider_id}.")
```

3. CLI Arguments

Argument	Required	Description
--video_provider_from	Yes	vonage or openvidu (case-insensitive).
--video_provider_to	Yes	Must be the opposite direction: vonage to openvidu, or openvidu to vonage.
--provider-id	Yes	providers.prov_id. Only sessions with this vsessions.prov_id are in scope. Must be greater than 0 and must exist in providers.
--dry-run	No	Log planned changes only. No database updates.
--client-emails	No	Comma-separated client login emails (contacts.cont_login_email), for example client1@gmail.com, client2@gmail.com. Matching is case-insensitive and spaces around commas are trimmed. Only sessions whose created_by_id points to a contact with one of those emails for the same --provider-id are migrated. Every listed email must resolve to at least one contact, otherwise the script exits with code 1. If omitted, all eligible sessions for the provider are considered with no filter on creator.

The parser is defined as follows:

```
parser.add_argument("--video_provider_from", required=True, ...)
parser.add_argument("--video_provider_to", required=True, ...)
parser.add_argument("--provider-id", type=int, required=True, metavar="PROV_ID", ...)
parser.add_argument("--dry-run", action="store_true", ...)
parser.add_argument("--client-emails", metavar="EMAIL_LIST", default=None, ...)
```

Direction is normalized and checked before any database writes take place:

```
video_provider_from = args.video_provider_from.strip().lower()
video_provider_to = args.video_provider_to.strip().lower()
supported_providers = {"vonage", "openvidu"}

if video_provider_from not in supported_providers:
    _fail("Unsupported provider. Use 'vonage' or 'openvidu'.")
if video_provider_to not in supported_providers:
    _fail("Unsupported target provider. Use 'vonage' or 'openvidu'.")
if video_provider_from == video_provider_to:
    _fail(
        "video_provider_from and video_provider_to must differ; nothing to migrate."
    )

expected_target = "openvidu" if video_provider_from == "vonage" else "vonage"
if video_provider_to != expected_target:
    _fail(
        "Invalid migration direction. Expected --video_provider_to "
        f"'{expected_target}' for --video_provider_from '{video_provider_from}'."
    )
```

The script exits with code 1 when any of these validation checks fail:

- A provider value other than vonage or openvidu was supplied.
- video_provider_from equals video_provider_to, so nothing would change.
- video_provider_to is not the opposite of video_provider_from.
- The --provider-id value is zero or negative, or the prov_id is unknown.

- No contact exists for a listed --client-emails address under that provider.
- Required configuration such as OPENVIDU_PUBLICURL or the Vonage keys is absent.

3.1 Client Emails Resolution

Emails are split on commas, stripped, lowercased, and de-duplicated. Each address must match `contacts.cont_login_email` for the same --provider-id. If any listed address has no matching contact, the script fails rather than migrating a partial set.

```
def _normalize_client_emails(raw: Optional[str]) -> list[str]:
    if raw is None or not raw.strip():
        return []
    seen: set[str] = set()
    out: list[str] = []
    for part in raw.split(","):
        norm = part.strip().lower()
        if not norm or norm in seen:
            continue
        seen.add(norm)
        out.append(norm)
    return out

def _contact_ids_for_client_emails(emails: list[str], provider_id: int) -> list[int]:
    rows = (
        db.session.query(Contact.id, func.lower(Contact.email))
        .filter(
            Contact.provider_id == provider_id,
            func.lower(Contact.email).in_(emails),
        )
        .all()
    )
    found_lower = {email_lc for _, email_lc in rows}
    missing = [e for e in emails if e not in found_lower]
    if missing:
        _fail(
            "No contact for --provider-id=%s with cont_login_email matching: %s"
            % (provider_id, ", ".join(missing))
        )
    return sorted({row[0] for row in rows})
```

4. Migration Behavior

4.1 Vonage To OpenVidu

This direction applies when --video_provider_from is vonage and --video_provider_to is openvidu. It adds the extra filter `LOWER(server) LIKE 'vonage%'` and resolves the target server from `ov.find_server()["OPENVIDU_PUBLICURL"]`, which must be present. For each matching row it sets `session.server` to the OpenVidu public URL, and sets `session.session_id` to "ses_" followed by `gen_random_session_id()`, unique per row.

```
q = q.filter(func.lower(VSession.server).like("vonage%"))
config = ov.find_server()
openvidu_server = config.get("OPENVIDU_PUBLICURL")
if not openvidu_server:
    _fail("OPENVIDU_PUBLICURL is missing; cannot migrate from vonage.")

sessions = q.all()
for session in sessions:
    old_server = session.server
    old_session_id = session.session_id
    if dry_run:
        new_sid_desc = "ses_<new unique random per row on apply>"
        # log planned server / session_id; no writes
    else:
        new_session_id = "ses_" + gen_random_session_id()
        session.server = openvidu_server
        session.session_id = new_session_id
```

A dry run does not call `gen_random_session_id()`. The log shows a placeholder for `session_id` instead, and apply mode generates a new id per row.

4.2 OpenVidu To Vonage

This direction applies when `--video_provider_from` is `openvidu` and `--video_provider_to` is `vonage`. The extra filter selects rows whose `LOWER(server)` does not start with `vonage`. Both `BaseConfig.VIDEO_PROVIDER` and `BaseConfig.VONAGE_API_KEY` must be present, though only their presence is checked. Note that the written server string is built from the `VONAGE_VIDEO_PROVIDER` constant rather than `BaseConfig.VIDEO_PROVIDER`. For each matching row, `session.server` becomes `VONAGE_VIDEO_PROVIDER-VONAGE_API_KEY-FLASK_ENV` and `session.session_id` is cleared to `None`.

```
q = q.filter(~func.lower(VSession.server).like("vonage%"))
if not BaseConfig.VIDEO_PROVIDER or not BaseConfig.VONAGE_API_KEY:
    _fail(
        "VIDEO_PROVIDER and VONAGE_API_KEY are required to migrate from openvidu."
    )
vonage_style_server = (
    f"{VONAGE_VIDEO_PROVIDER}-"
    f"{BaseConfig.VONAGE_API_KEY}-"
    f"{BaseConfig.FLASK_ENV}"
)

sessions = q.all()
for session in sessions:
    if not dry_run:
        session.server = vonage_style_server
        session.session_id = None
```

5. Dry Run

A dry run is enabled with the `--dry-run` flag. It behaves as follows:

- The query and logging are identical to apply mode.
- Each matching row is logged with the snapshot fields described in Section 6, followed by the planned server and `session_id` changes.
- No UPDATE statement and no commit are executed.
- The run ends with a dry-run summary line, and no rows are modified.

```
if dry_run:
    logger.info(
        "DRY-RUN COMPLETE | %s row(s) would be updated | no database changes made",
        count,
    )
    print(
        f"\nDry run finished: {count} row(s) would be updated. "
        f"See {LOG_PATH} for full detail.\n"
    )
    return

db.session.commit()
```

6. Logging

Logs are written to `migrate_vsession_server.log` at the project root, next to `pyproject.toml`. This follows the same pattern as `manage_db.py` and `db_operations.log`, using `setup_logger` from `scripts/python/script_utils.py`. Both a file handler and a console handler are attached, and the format includes a timestamp and a level.

```
LOG_FILENAME = "migrate_vsession_server.log"
LOG_PATH = os.path.join(_PROJECT_ROOT, LOG_FILENAME)
logger = setup_logger(
```

```

name="migrate_vsession_server",
log_file=LOG_PATH,
file_format="% (asctime)s | %(levelname)-8s | %(message)s",
console_format="% (levelname)s: %(message)s",
date_format="%Y-%m-%d %H:%M:%S",
)

```

Every per-row snapshot records the same ten model attributes: `id`, `session_id`, `session_name`, `scheduled_at`, `scheduled_end`, `state`, `server`, `started_at`, `ended_at`, and `created_by_id`.

Typical log content appears in the following order.

1. A banner showing the from and to video provider, the mode (dry run or apply), the log path, the provider scope (`prov_id`), and the command line (`argv`).
2. An eligibility summary listing the filters from Section 2, including `prov_id` and, when it is set, the `created_by_id` and client-email constraint.
3. When `--client-emails` is set, a line listing the resolved `vsession_created_by` IN (...) values, which are `contacts.cont_id` values.
4. A direction-specific plan covering which columns change and the resolved target server string.
5. One block per matching row. The block opens with the record number, such as Record 1 of N, together with `id` and `session_name`.
6. Within that block, a row snapshot of the current database values for all ten fields listed above.
7. Then the changes themselves, which cover `server` and `session_id` only, each shown as the previous value followed by the new value.
8. The block closes with a footer of either [DRY-RUN] or [UPDATED].
9. A final summary line. A dry run reports DRY-RUN COMPLETE with a count, while an apply reports COMMIT OK with a count.

7. Transaction And Error Handling

All work runs inside `FlaskAppManager()`, which provides the Flask app context, and every update runs in a single database transaction. On success in apply mode the script calls `db.session.commit()`, and the success message references the log file path. If an exception is raised, the script calls `db.session.rollback()`, logs the error, and exits with code 1.

```

with FlaskAppManager():
    try:
        # query + per-row updates
        if dry_run:
            return
        db.session.commit()
    except Exception as exc:
        db.session.rollback()
        logger.exception("Migration failed; transaction rolled back")
        _fail(f"Migration failed and was rolled back: {exc}")

```

The `_fail` helper always logs the message, prints it to `stderr`, and exits with status 1:

```

def _fail(message: str) -> NoReturn:
    logger.error(message)
    print(message, file=sys.stderr)
    raise SystemExit(1)

```

8. Usage

Run these commands from the repository root, replacing 42 with your own providers.prov_id. Always start with a dry run so the planned changes can be reviewed before anything is written.

Migrating from Vonage to OpenVidu as a dry run:

```
poetry run python -m scripts.python.migrate_vsession_server \
  --video_provider_from=vonage \
  --video_provider_to=openvidu \
  --provider-id=42 \
  --dry-run
```

The same provider, but restricted to sessions created by the listed client emails, where created_by_id maps to contacts for that prov_id:

```
poetry run python -m scripts.python.migrate_vsession_server \
  --video_provider_from=vonage \
  --video_provider_to=openvidu \
  --provider-id=42 \
  --client-emails="client1@gmail.com, client2@gmail.com" \
  --dry-run
```

Applying the same migration, with the --dry-run flag removed:

```
poetry run python -m scripts.python.migrate_vsession_server \
  --video_provider_from=vonage \
  --video_provider_to=openvidu \
  --provider-id=42
```

Migrating in the opposite direction, from OpenVidu to Vonage, first as a dry run and then as an apply:

```
poetry run python -m scripts.python.migrate_vsession_server \
  --video_provider_from=openvidu \
  --video_provider_to=vonage \
  --provider-id=42 \
  --dry-run

poetry run python -m scripts.python.migrate_vsession_server \
  --video_provider_from=openvidu \
  --video_provider_to=vonage \
  --provider-id=42
```

Ensure environment variables and the .env file are loaded so that the Flask app, the database, and ov.find_server() or the Vonage config resolve correctly when the script runs.

9. Related Code

Reference	Description
scripts/python/migrate_vsession_server.py	Full implementation.
lvdipatch.models.vsession.VSession	Provides server, session_id, prov_id, and created_by_id.
lvdipatch.models.contact.Contact	Provides email (cont_login_email) and provider_id.
lvdipatch.ov.find_server	Supplies the OpenVidu OPENVIDU_PUBLICURL value.
lvdipatch.util.gen_random_session_id	Supplies the OpenVidu session_id suffix.
lvdipatch.constants.VONAGE_VIDEO_PROVIDER	Supplies the Vonage server prefix.

10. Architecture Overview

The migration workflow runs in five phases:

1. Input validation, covering the provider values, the direction, and --provider-id.
2. Session discovery, which runs the eligibility query from Section 2.
3. Direction validation, which resolves the target server and checks required config.
4. Update processing, which applies the per-row changes or logs them in a dry run.
5. Transaction commit, or a rollback if any step raised an exception.

11. Troubleshooting Guide

Issue	Cause	Resolution
OPENVIDU_PUBLICURL missing	Environment variable not found	Update configuration
No contact found	Email does not exist under provider	Validate client emails
Transaction rollback	Database exception	Review logs